HOTEL RESERVATION SYSTEM

Sprint 01 Plan

Project	Hotel Reservation System
Document	Sprint 01 Plan
Document ID	SPRINT-001
Version	1.0
Status	Ready to Start
Owner	Delivery Team
Last Updated	2026-09-13

Official readable project artifact. Editable source is maintained under `docs/_source/`.

Contents

- Document control
- Sprint metadata
- Sprint Goal
- Sprint backlog - selected Product Backlog Items
- Delivery approach
- Implementation task register
  - S1-T01 - Repository and minimal application skeleton
  - S1-T02 - PostgreSQL persistence baseline and rooms domain model
  - S1-T03 - Public room catalogue REST API
  - S1-T04 - Room catalogue and details UI
  - S1-T05 - Availability domain rules and room blocking foundation
  - S1-T06 - Availability REST API
  - S1-T07 - Availability search UI
  - S1-T08 - Reservation persistence and database-level overlap protection
  - S1-T09 - Conflict-safe reservation creation REST flow
  - S1-T10 - Guest reservation form and booking confirmation UI
  - S1-T11 - Integrated booking tests and hardening
  - S1-T12 - Railway staging integration and Sprint Review readiness
- Capacity-based pull-in rule for PBI-005
- Sprint dependencies
- Sprint risks and responses
- Sprint 1 test strategy
  - Domain/integration priority
  - End-to-end priority
  - Security validation
- Definition of Done
- Integration strategy
- Sprint Review demonstration plan
- Expected Sprint Increment
- Progress register

Document control

Version	Date	Change	Author
1.0	2026-09-13	Initial Sprint 1 plan and task register	Product & Engineering

Sprint metadata

Field	Value
Sprint	Sprint 01
Duration	2 weeks (planning assumption)
Team assumption	1 full-time full-stack developer
Historical velocity	None - this is Sprint 1
Sprint status	Ready to Start
Primary delivery target	Inspectable Railway staging increment by Sprint end

Sprint Goal

> **Enable a guest to discover a room type, search real availability, and create a conflict-safe Pay-at-Hotel reservation with a booking reference using an integrated deployed application.**

This Sprint is valuable because it proves the highest-risk product path early: turning hotel inventory into a trustworthy direct reservation without double booking. The Sprint intentionally prioritizes reservation correctness and integration over marketing breadth, packages, or advanced administration.

Sprint backlog - selected Product Backlog Items

PBI	Outcome	Priority	Relative size	Sprint role
PBI-001	Guest can browse room types and room details	Critical	M	Selected

PBI	Outcome	Priority	Relative size	Sprint role
PBI-002	Guest can search real room-type availability	Critical	L	Selected
PBI-003	Guest can create a conflict-safe room reservation	Critical	L	Selected
PBI-004	Guest receives a safe booking confirmation view	Critical	S	Selected
PBI-005	Staff can securely sign in and inspect reservations	Critical	L	**Capacity-based pull-in candidate only**

PBI-005 must not be pulled in if doing so weakens availability correctness, reservation integrity, testing, or staging deployment quality.

Delivery approach

The Sprint is delivered as a vertical booking slice:

```
repository/app shell
    -> room inventory model
    -> room catalogue API/UI
    -> availability rules/API/UI
    -> reservation integrity + transaction
    -> booking form/confirmation
    -> integrated tests
    -> staging inspection
```

Technical foundation work exists only to enable this vertical path. The Sprint is not divided into separate frontend, backend, database, QA, and DevOps phases.

Implementation task register

Task state uses:

Not Started -> In Progress -> Review -> Done

If a task materially changes during the Sprint, the task notes/revision history should record why. The Sprint Backlog remains a living Developers' plan; task changes are allowed as long as the Sprint Goal is not endangered.

S1-T01 - Repository and minimal application skeleton

Status: Not Started
Supports: Sprint delivery foundation for PBI-001/PBI-002
Objective: Establish the professional npm-workspaces repository and the minimum Next.js/ Express application boundaries needed to begin the booking slice.

Why it exists

The project needs stable application and modular-monolith boundaries before persistence/ domain behavior is introduced. This task proves that the web application and API can run independently in one recognizable monorepo and communicate through configuration.

Scope

- npm workspace repository.
- `apps/web`.
- `apps/api`.
- Next.js + TypeScript frontend shell.
- Express + TypeScript backend shell.
- Feature-first Modular Monolith shape.
- `apps/api/src/modules/health`.
- `apps/api/src/config` only to the extent currently required.
- `apps/api/src/app.ts` for Express composition.
- `apps/api/src/server.ts` for process/server startup.
- `GET /health`.
- Frontend -> configured API health integration.
- Root workspace scripts/metadata needed to install, run, and build the two apps.
- Safe example environment configuration and Git ignore rules.

Explicit exclusions

- PostgreSQL.
- ORM/query builder/database driver architecture.
- Database migrations or seeds.
- `RoomType` or `Room`.
- Reservations or availability logic.
- Authentication/authorization.
- Packages.
- Email.
- Media.
- Future `shared/` abstractions.
- Empty future business modules.
- Empty `packages/` monorepo abstractions.

- Railway deployment implementation.
- CI workflow implementation.

Decisions to make during the task

- Pin/document the expected Node.js runtime version.
- Establish the npm-workspaces root configuration.
- Use web port `3000` and API port `4000` locally.
- Keep `/api/v1` reserved for future business APIs; `/health` remains an operational endpoint.
- Establish environment-variable naming without exposing server secrets to the browser.

Acceptance criteria

- Repository is a single npm-workspaces monorepo.
- `apps/web` is an independent Next.js TypeScript app.
- `apps/api` is an independent Express TypeScript app.
- Backend shell is module-oriented rather than global layer-first.
- `app.ts` constructs the Express application; `server.ts` owns process/server startup.
- `modules/health` owns the `/health` behavior without unnecessary service/repository layers.
- Both applications start locally and build successfully.
- Frontend can reach the API health endpoint through configuration.
- Real environment files/secrets are ignored by Git; safe example config contains no credentials.
- No speculative business modules, shared abstractions, database code, or future-Sprint functionality is introduced.

Required tests/checks

- Start API and request `/health` directly.
- Start Next.js and verify the web application loads.
- Verify frontend-to-API health communication.
- Stop the API and confirm the frontend handles failure without an uncontrolled crash.
- Run production builds for both applications.
- Inspect Git status to ensure secrets/generated output are not accidentally tracked.

Security / quality considerations

- Never place secrets in client-exposed Next.js environment variables.
- Do not commit `.env` files containing credentials.
- Do not use permissive production CORS/security configuration merely to solve local development.
- Avoid overengineering the health module or creating future architecture by placeholder.

Definition of Done checks

- Local run/build passes.
- Configuration is understandable and safe.
- Application boundaries are clear.
- Repository is reviewable by an experienced Node.js developer.
- No blocking errors remain for the next task.

S1-T02 - PostgreSQL persistence baseline and rooms domain model

Status: Not Started
Supports: PBI-001, PBI-002
Objective: Introduce PostgreSQL persistence, repeatable migrations, representative seed data, and the minimum `rooms` module domain required for room types and physical rooms.

Why it exists

Availability must be calculated from real physical inventory. The project needs a deliberate persistence approach and room model before building public catalogue/availability behavior.

Scope

- Select the minimal PostgreSQL access/migration approach after reviewing transaction/constraint needs.
- Introduce database configuration and migration mechanism.
- Create `RoomType` and `Room` persistence with capacity, active state, and base nightly EGP price on the appropriate owner.
- Seed representative development inventory: Deluxe Double (2 physical rooms) and Family Room (1 physical room).
- Establish database ownership within the feature-first modular monolith without global mixed-domain repositories.

Acceptance criteria

- Clean database can be migrated repeatably.
- Room types and physical rooms are related correctly.
- Active state, capacity, and non-negative/valid base pricing are constrained appropriately.
- Seed data can be recreated predictably for development/testing.
- No reservation or authentication functionality is introduced.

Required tests

- Migration from empty database.
- Seed execution.

- Referential-integrity checks.
- Basic data constraint checks.

Security / quality considerations

- `DATABASE_URL` remains server-side and uncommitted.
- Schema choices should support later reservation integrity without premature rate-plan design.

S1-T03 - Public room catalogue REST API

Status: Not Started
Supports: PBI-001
Objective: Expose guest-safe active room-type catalogue/detail behavior from the `rooms` module.

Acceptance criteria

- Public API returns active room types with guest-safe fields.
- Internal physical room numbers/codes are not exposed.
- Invalid identifiers produce controlled API errors.
- Data comes from PostgreSQL, not hardcoded API constants.

Required tests

- Active/inactive filtering.
- Detail lookup success/not-found.
- Public response field set excludes physical room identifiers.

Security / quality considerations

- Server-side validation and consistent error handling.
- Keep `rooms` domain logic within the module; do not create global business layers.

S1-T04 - Room catalogue and details UI

Status: Not Started
Supports: PBI-001
Objective: Build the minimal responsive guest UI for browsing room types and opening room details.

Acceptance criteria

- Catalogue and detail routes consume the API.

- Name, description, amenities/representative content, capacity, and EGP nightly price are clear.
- Public UI never exposes physical room numbers.
- Basic loading, error, and empty states are handled.
- Implemented views are usable at desktop/tablet/mobile widths.

Required tests/checks

- Basic UI/component tests where valuable.
- Manual responsive and error-state validation.
- API integration smoke check.

S1-T05 - Availability domain rules and room blocking foundation

Status: Not Started
Supports: PBI-002
Objective: Implement the server-side definition of room availability for `[check-in, check-out)` and introduce `RoomBlock` only if needed for the engine.

Acceptance criteria

- Date validation and occupancy/capacity rules are explicit.
- Same-day checkout/check-in is allowed.
- Partial, complete, and enclosing overlaps exclude inventory.
- Availability is based on physical rooms, not a room-type boolean flag.
- Any `RoomBlock` model uses compatible date semantics.

Required tests

- Valid availability.
- Partial overlap.
- Complete overlap.
- Enclosing overlap.
- Same-day turnover.
- Guest capacity boundaries.
- Invalid/past date policy.

Security / quality considerations

- Backend is authoritative.
- Avoid leaking physical room identity in public results.

S1-T06 - Availability REST API

Status: Not Started
Supports: PBI-002
Objective: Expose room-type availability search through the versioned REST API.

Acceptance criteria

- API accepts check-in, check-out, and guest count.
- Inputs are validated server-side.
- Response returns only room types with at least one eligible physical room for the whole stay.
- Public response remains guest-safe.

Required tests

- API integration tests covering the domain overlap/capacity matrix.
- Invalid input and controlled error tests.

S1-T07 - Availability search UI

Status: Not Started
Supports: PBI-002
Objective: Let the guest search dates/occupancy and see server-authoritative available room types.

Acceptance criteria

- Guest can enter check-in, check-out, and number of guests.
- UI provides helpful client-side validation without replacing backend validation.
- Results reflect the API response and handle no-results/error/loading states.
- Selecting a result can continue toward the reservation flow.

Required tests/checks

- Valid search.
- Invalid input UX.
- No-availability state.
- API failure state.

S1-T08 - Reservation persistence and database-level overlap protection

Status: Not Started
Supports: PBI-003
Objective: Establish reservation/contact/allocation/pricing snapshot persistence and the database invariant that prevents overlapping inventory-consuming reservations for one physical room.

Acceptance criteria

- Reservation stores room type reference, internal physical room allocation, dates, contact snapshot, Pending status, Unpaid/Pay-at-Hotel payment status, price snapshot, and unique booking reference.
- Date semantics remain `[check-in, check-out)`.
- Database prevents overlapping inventory-consuming reservations for the same physical room.
- Historical reservation total does not depend on later room-type price changes.

Required tests

- Constraint/invariant tests for overlap matrix.
- Same-day turnover allowed.
- Price snapshot persistence.
- Booking reference uniqueness.

Security / quality considerations

- Avoid unnecessary guest PII.
- Do not log sensitive reservation contact fields.

S1-T09 - Conflict-safe reservation creation REST flow

Status: Not Started
Supports: PBI-003
Objective: Implement the atomic backend reservation use case from validated request to committed guest-safe confirmation data.

Acceptance criteria

- Server validates dates, capacity, and guest/contact input.
- Server ignores/rejects client attempts to control physical room, nightly price, or total.
- Availability is rechecked during creation.
- Eligible physical room is allocated atomically.
- Nights/price are calculated server-side and snapshotted.

- Pending reservation immediately consumes inventory.
- Public result contains booking reference and guest-safe fields only.

Required tests

- Successful creation.
- No availability.
- Invalid capacity/date/contact.
- Correct nights and price calculation.
- Historical price snapshot.
- Two concurrent overlapping requests for one physical room: exactly one succeeds.
- Transaction rollback on failure.

S1-T10 - Guest reservation form and booking confirmation UI

Status: Not Started
Supports: PBI-003, PBI-004
Objective: Complete the guest vertical slice from selected available room type through contact entry to safe confirmation.

Acceptance criteria

- Guest reviews selected room type, dates, nights, guest count, and server-calculated price context.
- Required contact data can be entered and submitted.
- Successful response displays booking reference, room type, dates, guests, nights, total EGP, and Pay at Hotel.
- Internal physical room identity is never shown.
- Failed submission never renders success.

Required tests/checks

- Happy path.
- Validation errors.
- Inventory-lost-between-search-and-submit path.
- Confirmation privacy/field-set check.

S1-T11 - Integrated booking tests and hardening

Status: Not Started
Supports: PBI-001 to PBI-004
Objective: Prove the complete Sprint Goal and close gaps found through integration testing.

Acceptance criteria

- Automated integration suite covers critical domain rules.
- End-to-end booking journey works with representative fixtures.
- Concurrent booking invariant is repeatable.
- No unresolved Critical/Blocker defect remains.
- Error/log output does not expose secrets or unnecessary guest PII.

Required tests

- Full date-boundary matrix.
- Capacity matrix.
- Server-side price and snapshot behavior.
- Pending inventory consumption.
- Concurrency race.
- End-to-end browse -> search -> reserve -> confirm.
- Security validation of untrusted client price/room inputs.

S1-T12 - Railway staging integration and Sprint Review readiness

Status: Not Started
Supports: Sprint Goal / Definition of Done
Objective: Deploy the integrated Sprint increment to an inspectable Railway staging environment and verify the review scenario.

Acceptance criteria

- Web, API, and PostgreSQL staging services are integrated.
- Staging configuration/secrets are provided through environment configuration.
- Health endpoint is usable operationally.
- Database migrations/seed strategy works safely for staging.
- Critical booking smoke test passes in staging.
- Sprint Review demonstration can be performed without local-only dependencies.

Required tests/checks

- Staging health check.

- Browse/search/reserve/confirm smoke test.
- One-room concurrency scenario demonstrated/tested against the intended database behavior.
- Logs inspected for useful errors and PII/secret leakage.

Capacity-based pull-in rule for PBI-005

Only after S1-T01 through S1-T12 are sufficiently complete and the Sprint Goal is not at risk may the Developer consider refining/pulling PBI-005. The Sprint Goal is more important than maximizing item count.

Sprint dependencies

- npm/Node.js development environment.
- Railway account/project access by the staging task.
- PostgreSQL availability for local/integration testing from S1-T02 onward.
- Representative development fixtures; final hotel content is not required for Sprint 1.
- Client final content/assets can proceed in parallel without blocking the booking engine.

Sprint risks and responses

Risk	Impact	Response
Availability correctness	Guest sees false inventory or cannot book valid inventory	Define `[check-in, check-out)` explicitly; integration-test overlap boundaries against PostgreSQL.
Concurrent reservations	Double booking	Database-level overlap protection plus concurrency integration test where exactly one request succeeds.
Date-boundary errors	Off-by-one nights/turnover failures	Centralize date semantics and test same-day checkout/check-in, partial, complete, and enclosing ranges.
Pricing correctness	Incorrect guest total / historical amount changes	Calculate server-side; persist price snapshot; test price change after booking.
Scope expansion	Sprint Goal endangered by packages/marketing/admin work	Keep selected PBIs fixed around the core booking slice; PBI-005 only as pull-in.

Risk	Impact	Response
Overengineering	Developer time spent on abstractions rather than product outcome	Add modules/layers/shared code only when current task requires them.
Insufficient testing	Hidden integrity defects discovered late	Write domain/integration tests alongside implementation, especially for database constraints and concurrency.
Configuration/security mistakes	Secrets exposure or unsafe public behavior	Environment-driven config, ignored secrets, guest-safe responses, server-side validation, log review.

Sprint 1 test strategy

Testing follows the risk profile rather than aiming for uniform coverage percentages.

Domain/integration priority

- Valid availability.
- Partial date overlap.
- Complete date overlap.
- Enclosing date ranges.
- Same-day checkout/check-in.
- Guest capacity boundaries.
- Invalid/past dates.
- Correct number-of-nights calculation.
- Server-side price calculation.
- Historical price snapshot.
- Pending reservation consumes inventory.
- Concurrent reservation attempts against the one remaining physical room: exactly one succeeds.

End-to-end priority

- Browse room types.
- Search dates/occupancy.
- Select available room type.
- Enter guest contact data.
- Submit booking.
- Receive booking reference and safe confirmation.

- Search again and observe reduced inventory.

Security validation

- Client-supplied physical room ID, nightly price, and total are not authoritative.
- Malformed input is rejected safely.
- Public responses do not expose physical room numbers.
- Secrets are not tracked or logged.
- Guest PII is not unnecessarily logged.

Definition of Done

A selected PBI is Done only when all applicable criteria below hold:

- Acceptance criteria are satisfied in the integrated application.
- Server-side validation and authorization exist where required.
- Relevant automated unit/integration tests pass.
- Booking/availability changes include database-level integrity tests.
- Relevant loading, error, empty, and validation UX states are implemented.
- No unresolved Critical/Blocker defects remain.
- Database migrations are repeatable and tested against a clean database.
- No secrets/credentials are committed.
- Admin operations, when present, are inaccessible without valid authorization.
- Guest PII is not unnecessarily logged or publicly exposed.
- CI checks pass once CI is introduced.
- Code receives deliberate review; with one developer, a structured self-review/diff review is required until a second reviewer exists.
- Relevant operational logging is present.
- The change is integrated and deployed to the Sprint’s inspectable target environment.
- API/documentation notes are updated when future development would otherwise be ambiguous.
- The Increment remains usable together with everything previously Done.

A local-only feature is not Done for the Sprint Increment.

Integration strategy

- Keep branches short-lived and integrate frequently.
- Keep the main branch deployable once the CI/deployment baseline exists.
- Database schema changes are repeatable migrations, not undocumented manual changes.
- Frontend/API changes for one outcome should be integrated together early rather than merged at the end of the Sprint.

- Use representative fixtures until final hotel content arrives.
- Staging is the Sprint’s inspectable integration environment.

Sprint Review demonstration plan

1. Open the deployed staging site and browse seeded room types.
2. Search a free date range and demonstrate correct available room types.
3. Create a reservation and show the booking reference and guest-safe confirmation.
4. Search the same interval again and demonstrate reduced inventory.
5. Use a room type with one physical room and show that two competing overlapping booking attempts cannot both succeed.
6. Demonstrate same-day checkout/check-in turnover behavior.
7. Show that changing the current room-type price does not alter the historical reservation total (if the review environment safely supports the demonstration).
8. Capture stakeholder feedback and adapt the Product Backlog without treating current details as frozen.

Expected Sprint Increment

At Sprint end, the target integrated behavior is:

```
Guest
  -> browses room types
  -> selects dates and guests
  -> searches real availability
  -> selects a room type
  -> enters booking/contact details
  -> backend atomically allocates physical inventory
  -> reservation and price snapshot are stored
  -> booking reference is created
  -> guest-safe confirmation is displayed
```

The broader marketing site, packages, confirmation email, and advanced admin management are not required to prove the Sprint Goal unless separately pulled in after the core increment is safe.

Progress register

Task	Initial status	Current status	Notes
S1-T01 Repository/ application skeleton	Not Started	Not Started	First development task; current bounded task.
S1-T02 PostgreSQL + rooms persistence	Not Started	Not Started	Starts only after T01 review.

Task	Initial status	Current status	Notes
S1-T03 Room catalogue API	Not Started	Not Started	-
S1-T04 Room catalogue UI	Not Started	Not Started	-
S1-T05 Availability domain rules	Not Started	Not Started	-
S1-T06 Availability API	Not Started	Not Started	-
S1-T07 Availability UI	Not Started	Not Started	-
S1-T08 Reservation persistence/invariant	Not Started	Not Started	-
S1-T09 Reservation creation API	Not Started	Not Started	-
S1-T10 Reservation/ confirmation UI	Not Started	Not Started	-
S1-T11 Integrated tests/ hardening	Not Started	Not Started	-
S1-T12 Railway staging/ review readiness	Not Started	Not Started	-